

Spinning Up Modern: Modernization Report

Gemini CLI Agent

April 2026

1 Introduction

This document details the modernization efforts applied to the OpenAI Spinning Up in Deep RL repository. The project has been updated to support contemporary Python environments, latest deep learning frameworks (PyTorch 2.x and TensorFlow 2.x), and modern RL environment standards.

2 Core Modernization Pillars

2.1 Environment API: Gymnasium

The repository has transitioned from the legacy `gym` library to `gymnasium`.

- Updated `env.step()` to handle the new return signature: `obs`, `reward`, `terminated`, `truncated`, `info`.
- Updated `env.reset()` to handle: `obs`, `info`.
- Replaced all `import gym` with `import gymnasium as gym`.
- Standardized default environments to latest versions: `HalfCheetah-v5` and `CartPole-v1`.

2.2 Framework Updates

2.2.1 PyTorch 2.x

- Integrated `torch.compile` for optimized execution graphs.
- Added support for GPU/CUDA device placement.
- Fixed device-specific errors by ensuring proper `.detach().cpu().numpy()` calls.

2.2.2 TensorFlow 2.x

- Complete migration from TensorFlow 1.x (Session/Placeholder based) to TensorFlow 2.x using Keras models and Eager Execution.
- Utilized `tf.GradientTape` for gradient calculation.
- Implemented custom MPI-aware optimizers compatible with TF2.
- Resolved initialization factory registration and TensorRT conflicts.

2.3 Device Management and GPU Acceleration

A framework-agnostic device management system has been implemented in `spinup.utils.device_utils`. This module ensures that both PyTorch and TensorFlow 2.x utilize available hardware acceleration.

2.3.1 NVIDIA Library Dependency Resolution

A significant challenge in modernizing the repository was the version mismatch between pip-installed NVIDIA libraries (e.g., cuDNN 9) and the specific versions expected by TensorFlow 2.15 (e.g., cuDNN 8). To resolve this, a dynamic library mapper was implemented. This strategy:

- Scans the user's `site-packages` for NVIDIA-provided shared objects.
- Creates version-mapped symbolic links in a localized `tf_gpu_libs` directory.
- Dynamically injects these paths into `LD_LIBRARY_PATH` at runtime.

This ensures that the repository remains "plug-and-play" across diverse Linux environments with standard NVIDIA drivers.

3 Usage and Performance

3.1 Running on GPU

To run an algorithm on the GPU, simply use the provided CLI. The backends will automatically detect and use the most capable device.

```
1 # Run PPO with PyTorch on GPU
2 python -m spinup.run ppo --env CartPole-v1
3 # Run PPO with TensorFlow 2 on GPU
4 python -m spinup.run ppo_tf2 --env CartPole-v1
```

4 Refactoring and Code Quality

- **Type Hinting:** Comprehensive Python type hints have been added to all algorithm functions and core classes.
- **Standalone Architectures:** Maintained the original design goal of standalone algorithm implementations while ensuring they are modern and efficient.
- **Robust Logging:** Updated `EpochLogger` to handle edge cases like empty data batches at epoch boundaries.

5 Verification

The modernization was verified through a comprehensive test suite covering all core algorithms (VPG, PPO, DDPG, TD3, SAC, TRPO) across both backends.

- **PyTorch Tests:** 100% Pass.
- **TF2 Tests:** 100% Pass.

6 Conclusion

The `spinningup-modern` repository is now a state-of-the-art educational resource for Deep Reinforcement Learning, optimized for modern hardware and software ecosystems.